

Buildroot build imgs

Use buildroot to build the community system

- Source code warehouse address <https://github.com/DongshanPI/buildroot-external-dongshanpione>
- build step

```
git clone https://github.com/DongshanPI/buildroot-external-dongshanpione
cd buildroot-external-dongshanpione
make
```

After the compilation is completed, the output files are stored in the directory under the build outputs directory files are as follows.

```
book@virtual-machine:~/DongshanPiOne/buildroot-external-
dongshanpione/outputs$ ls -lh
total 25M
-rw-r--r-- 1 book book 44K Apr 1 03:20 dongshanpione-ipl
-rw-r--r-- 1 book book 3.4M Apr 1 03:20 dongshanpione-kernel.fit
-rw-r--r-- 1 book book 4.4M Apr 1 03:20 dongshanpione-kernel-rescue.fit
-rw-r--r-- 1 book book 3.5M Apr 1 03:20 dongshanpione-rootfs.squashfs
-rw-rw-r-- 1 book book 13M Apr 1 03:20 dongshanpione-ubi.img
-rw-r--r-- 1 book book 465K Apr 1 03:20 dongshanpione-u-boot.img
-rw-rw-r-- 1 book book 2.0K Apr 1 03:20 GCIS.bin
-rw-rw-r-- 1 book book 22K Apr 1 03:20 IPL.bin
```

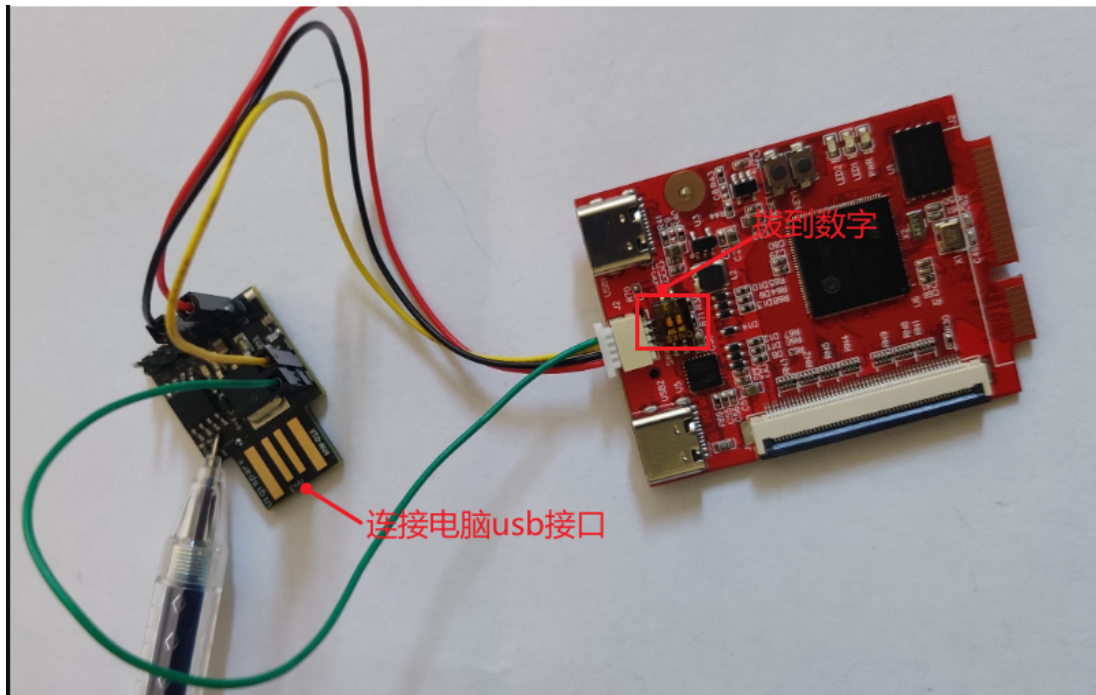
Next, the programming operation is performed.

run rescue system

1. Program u-boot-2022

- Use the open source programming tool to program the first-stage bootloader first.
- For a more detailed description of open source community tools, please see, the download and installation method is also on this page.
- Community Edition programming tool connection introduction
<https://item.taobao.com/item.htm?id=665728385852>
- After connecting the programming tool to the programming line, connect the programming device to the programming interface of the development board, and switch the DIP switch

to the programming mode (the white DIP DIP switch to the non-NO direction).



After that, we press the key on the main board `Reset` and execute the following programming command at the same time, the first sentence `sudo SNANDer -p mstarddc -c /dev/i2c-0:49 -e` is the entire erase flash command, it is recommended to execute. Note: The programming operation is performed in ubuntu-18, and the programming tool needs to be mounted in the ubuntu system. After the mounting is successful, the system will automatically install the driver module. At this time, the device manager will have an `/dev/i2c-0` additional device node. The `-l` parameter specifies the length (size) of the programming file. If you have modified the size of the source code, please confirm whether the size is correct.

Before flashing, please make sure you have entered the directory of the image that needs to be flashed.

Note: Please do not open the serial port tool before or during programming, otherwise the programming will fail. * The following are the detailed programming commands, which need to be executed in sequence.

```
sudo SNANDer -p mstarddc -c /dev/i2c-0:49 -e
sudo SNANDer -p mstarddc -c /dev/i2c-0:49 -w GCIS.bin
sudo SNANDer -p mstarddc -c /dev/i2c-0:49 -a 0x140000 -l 0x6000 -w IPL.bin
sudo SNANDer -p mstarddc -c /dev/i2c-0:49 -a 0x200000 -l 0xB000 -w
dongshanpione-ipl
```

After completing the first part of programming, please unplug the programmer first, use tweezers to adjust the DIP switch to `ONKE` the letter side, then use the MobaXterm tool to open the serial terminal, after opening the terminal, press the `reset` reset button, you can see the following picture The log startup information shown.

```

IPL g76cd9eb
D-1e
HW Reset
miupll_233MHz
MIU0 zq=0x003b
miu_bw_set
utmi_1_init done
utmi_2_init done
utmi_3_init done
usbpll init done.....
cpupll init done
SPI 54M
clk_init done
P1 USB_rterm trim=0x0000
P1 USB_HS_TX_CURRENT trim=0x0000
P2 USB_rterm trim=0x0000
P2 USB_HS_TX_CURRENT trim=0x0001
P3 USB_rterm trim=0x0000
P3 USB_HS_TX_CURRENT trim=0x0001
PM_vol_bgap trim=0x0002
GCR_SAR_DATA trim=0x0193
ETH 10T output swing trim=0x0010
ETH 100T output swing trim=0x0010
ETH RX input impedance trim=0x0000
ETH TX output impedance trim=0x0001
MIPI_HS_RTERM trim=0x0001
MIPI_LP_RTERM trim=0x0000
128MB
BIST0_0001-OK
Enable MMU and CACHE
Load IPL_CUST from SPINAND
QUAD MODE ENABLE
unable to find IDX for part type:0001
[I]m7
Checksum OK

cpuid: 410fc075, mstar chipid: f0
IPL image is broken
normal power on
wrote 1f206540 <- 4bc7 was 4bc7, readback 4bc7
wrote 1f206544 <- 0037 was 0037, readback 0037
wrote 1f206548 <- b3d5 was 0000, readback b3d5
wrote 1f20654c <- 0043 was 0000, readback 0043
wrote 1f206560 <- 0001 was 0000, readback 0001
wrote 1f206554 <- 0006 was 0006, readback 0006
wrote 1f20655c <- 0008 was 0008, readback 0008
wrote 1f206564 <- 1000 was 0000, readback 1000
wrote 1f206550 <- 0000 was 0000, readback 0000
wrote 1f206550 <- 0001 was 0000, readback 0001
wrote 1f20442c <- 0001 was 0000, readback 0001
wrote 1f203ddc <- 0004 was 0000, readback 0004
wrote 1f203dd4 <- 4004 was 0000, readback 4004
wrote 1f203dd8 <- 0001 was 0000, readback 0001
wrote 1f203dc0 <- 0000 was 0000, readback 0000
wrote 1f203dc0 <- 8000 was 0000, readback 8000
delaying for 100
readback: 4278

```

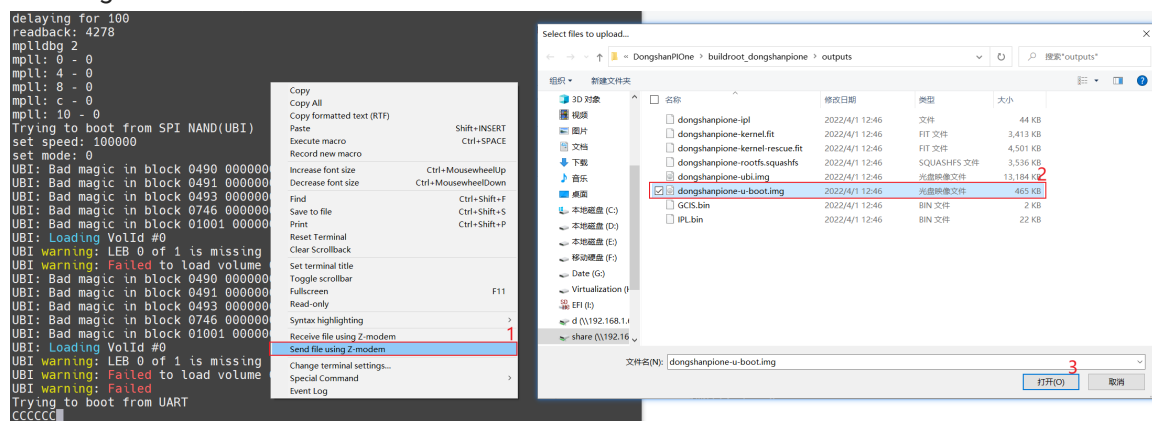
```

mplldbg 2
mp1l: 0 - 0
mp1l: 4 - 0
mp1l: 8 - 0
mp1l: c - 0
mp1l: 10 - 0
Trying to boot from SPI NAND(UBI)
set speed: 100000
set mode: 0
UBI: Bad magic in block 0490 00000000
UBI: Bad magic in block 0491 00000000
UBI: Bad magic in block 0493 00000000
UBI: Bad magic in block 0746 00000000
UBI: Bad magic in block 01001 00000000
UBI: Loading VolId #0
UBI warning: LEB 0 of 1 is missing
UBI warning: Failed to load volume 0
UBI: Bad magic in block 0490 00000000
UBI: Bad magic in block 0491 00000000
UBI: Bad magic in block 0493 00000000
UBI: Bad magic in block 0746 00000000
UBI: Bad magic in block 01001 00000000
UBI: Loading VolId #0
UBI warning: LEB 0 of 1 is missing
UBI warning: Failed to load volume 0
UBI warning: Failed
Trying to boot from UART
CC

```

Then we use the serial port to send the protocol Z-modem or Y-modem according to the prompt information, `u-boot.img` that is, the second part is `dongshanpione-u-boot.img` sent to the IPL program of Dongshan PI One, which is used to load and start into the u-boot command line.

We have taken MobaXterm as an example. When the system starts to print a `C` string, you can refer to the subordinate operations on the current serial port terminal, right-click to pop up the menu option, move to the ****Send file using Z-modem**** item, and then pop up a dialog box. Select the `dongshanpione-u-boot.img` image and wait for the sending to complete, as shown in the figure below.



```
CCCSending: dongshanpione-u-boot.img
```

Expect to wait about 1 minute. After the transfer is completed, ****Transfer complete**** will be prompted to indicate completion, and then the U-Boot image will be automatically started.

```
Transfer complete
Loaded 475852 bytes

U-Boot 2022.04-rc2 (Mar 08 2022 - 22:03:20 +0000)

DRAM: 128 MiB
wrote 1f206540 <- 4bc7 was 4bc7, readback 4bc7
wrote 1f206544 <- 0037 was 0037, readback 0037
wrote 1f206548 <- b3d5 was b3d5, readback b3d5
wrote 1f20654c <- 0043 was 0043, readback 0043
wrote 1f206560 <- 0001 was 0001, readback 0001
wrote 1f206554 <- 0006 was 0006, readback 0006
wrote 1f20655c <- 0008 was 0008, readback 0008
wrote 1f206564 <- 1000 was 1000, readback 1000
wrote 1f206550 <- 0000 was 0001, readback 0000
wrote 1f206550 <- 0001 was 0000, readback 0001
wrote 1f20442c <- 0001 was 0001, readback 0001
wrote 1f203ddc <- 0004 was 0004, readback 0004
wrote 1f203dd4 <- 4004 was 4004, readback 4004
wrote 1f203dd8 <- 0001 was 0001, readback 0001
wrote 1f203dc0 <- 0000 was 8000, readback 0000
wrote 1f203dc0 <- 8000 was 0000, readback 8000
delaying for 100
readback: 4277
mp11 here!
mp11 here xxxx!
checking 26f84528 26f84528
found parent at 0
mp11 is already running
Core: 43 devices, 16 uclasses, devicetree: fit
WDT: Started watchdog@6000 with servicing (60s timeout)
MMC: MSC: 0
Loading Environment from UBI... set speed: 100000
set mode: 0
Volume env not found!

** Unable to read env from UBI:env **
In: uart@221000
Out: uart@221000
Err: uart@221000
Net: mstar pinctrl g 0, f 0
found selector
checking 26f85630 26f85630
found parent at 0
emac patches
rx ring 26fcdb40
eth0: emac@2a2000
Hit any key to stop autoboot: 0
Volume kernel not found!
```

```
Wrong Image Format for bootm command
ERROR: can't get kernel image!
=>
```

After entering, we can run the smallest Linux rescue system first, and after entering the system, burn each part of the image into the nand flash.

2. Run the kernel system

Next, we need to load and run the kernel through the network tftp, or download and run the kernel through the serial port.

Use network tftp to download (recommended)

- First of all, we need to configure the tftp service of the ubuntu system, and copy the `dongshanpione-kernel-rescue.fit` files to the configured tftp directory. After that, please remember that your ubuntu host IP address, refer to the subordinate command to run, as shown in the following example. As shown, the host IP address of our ubuntu tftp server storing the kernel image is 192.168.1.133, you need to modify it according to your situation.

```
=> setenv loadaddr 0x22000000; dhcp 192.168.1.133:dongshanpione-kernel-
rescue.fit;bootm 0x22000000#ssd202d-dongshanpione
```

After the execution is complete, it will automatically download your kernel image file from the tftp directory of ubuntu, and then start it, similar to the following print message output

```
emac@2a2000: PHY present at 0
PHY reset timed out
phy power up
Doing phy power up
emac@2a2000: Starting autonegotiation...
emac@2a2000: Autonegotiation complete
emac@2a2000: link up, 100Mbps full-duplex (lpa: 0xcde1)
BOOTP broadcast 1
BOOTP broadcast 2
BOOTP broadcast 3
DHCP client bound to address 192.168.1.53 (793 ms)
Using emac@2a2000 device
TFTP from server 192.168.1.133; our IP address is 192.168.1.53
Filename 'dongshanpione-kernel-rescue.fit'.
Load address: 0x22000000
Loading: #####
#####
#####
#####
#####
3.4 MiB/s
done
```

```
Bytes transferred = 4607292 (464d3c hex)
## Loading kernel from FIT Image at 22000000 ...
Using 'ssd202d-dongshanpione' configuration
Trying 'kernel-0' kernel subimage
  Description: unavailable
  Type:        Kernel Image
  Compression: uncompressed
  Data Start:   0x220000a4
  Data Size:    3403824 Bytes = 3.2 MiB
  Architecture: ARM
  OS:           Linux
  Load Address: 0x21800000
  Entry Point:   0x21800000
  Hash algo:     crc32
  Hash value:    dc0ebb3a
  Hash algo:     sha1
  Hash value:    881943efc7cf947392bffd3ae624733f2507bd35
Verifying Hash Integrity ... crc32+ sha1+ OK
## Loading ramdisk from FIT Image at 22000000 ...
Using 'ssd202d-dongshanpione' configuration
Trying 'ramdisk-0' ramdisk subimage
  Description: unavailable
  Type:        RAMDisk Image
  Compression: uncompressed
  Data Start:   0x2234c650
  Data Size:    1147180 Bytes = 1.1 MiB
  Architecture: ARM
  OS:           Linux
  Load Address: unavailable
  Entry Point:   unavailable
  Hash algo:     crc32
  Hash value:    c40c3d1c
  Hash algo:     sha1
  Hash value:    702d8f01c746bdfbfcaebd4572b95ecf32e3bf7c
Verifying Hash Integrity ... crc32+ sha1+ OK
## Loading fdt from FIT Image at 22000000 ...
Using 'ssd202d-dongshanpione' configuration
Trying 'fdt-1' fdt subimage
  Description: unavailable
  Type:        Flat Device Tree
  Compression: uncompressed
  Data Start:   0x22345c04
  Data Size:    27001 Bytes = 26.4 KiB
  Architecture: ARM
  Hash algo:     crc32
  Hash value:    4ed1ac67
  Hash algo:     sha1
  Hash value:    6209734a29f8a3c9f2ba1329135358d6bbda62d4
Verifying Hash Integrity ... crc32+ sha1+ OK
Booting using the fdt blob at 0x22345c04
Loading Kernel Image
Loading Ramdisk to 26e64000, end 26f7c12c ... OK
Loading Device Tree to 26e5a000, end 26e63978 ... OK

Starting kernel ...

[ 0.000000] Booting Linux on physical CPU 0x0
```

```
[ 0.000000] Linux version 5.17.0 (buildroot@buildroot) (arm-buildroot-
linux-musleabihf-gcc.br_real (Buildroot 2022.02-349-g1506ff275b) 11.2.0, GNU
ld (GNU Binutils) 2.37) #1 SMP Mon Mar 28 12:29:22 UTC 2022
```

After waiting for startup, you can enter the user name `root` to enter the system.

Send using serial protocol

- For students who have no network or inconvenient environment, you can use the serial port protocol to transmit the kernel image under u-boot. This method is very slow in transmission and is not recommended. We only need to enter the following command in the u-boot command line, then disconnect the secondary serial terminal, recreate a serial terminal with a baud rate of 460800, and then use the Z-modem command to transfer files according to the prompts.

```
setenv loadaddr 0x22000000; loady ${loadaddr} 460800; bootm
```

After the transmission is completed, according to the prompt, disconnect the current serial port terminal, then switch back to the original 115200 baud rate terminal, and press the keyboard `ESC` key to automatically load and start the system. The startup information is similar to the above logo.

3. Program the system to flash

According to the above operations, we have entered the rescue system of the kernel, and then continue to perform the following operations to burn the compiled system image to the onboard Nandflash chip.

1. Format to create a partition

- As shown in the command below, you need to format the Nandflash first and create a mirror partition of each part.

```
ubiformat /dev/mtd1
ubiattach -m 1
ubimkvol /dev/ubi0 -N uboot -s 1MiB -t static
ubimkvol /dev/ubi0 -N env -s 256KiB
ubimkvol /dev/ubi0 -N kernel -s 16MiB
ubimkvol /dev/ubi0 -N rescue -s 16MiB
ubimkvol /dev/ubi0 -N rootfs -m
```

2. Copy and program

- We can use the nfs network mount method to copy and burn these files.

```
mount -t nfs -o nolock 192.168.1.133:/home/book/nfs_rootfs /mnt
```

1. Program the rescue system

- Burn the u-boot image

```
ubiupdatevol /dev/ubi0_0 dongshanpione-u-boot.img
```

- Burn the rescue kernel image

```
ubiupdatevol /dev/ubi0_3 dongshanpione-kernel-rescue.fit
```

After the programming is completed, reset it again, enter the uboot command line, and execute the following commands to set the startup parameters, and the rescue system image can be automatically loaded from the partition just programmed and started.

```
setenv loadaddr 0x22000000; ubi readvol ${loadaddr} rescue 0x800000; bootm
```

2. Burn the kernel and root file system

- Burn the normal kernel image file

```
ubiupdatevol /dev/ubi0_2 dongshanpione-kernel.fit
```

- Flash root file system

```
ubiupdatevol /dev/ubi0_4 dongshanpione-rootfs.squashfs
```

- Set the startup parameters under uboot and save them in env, and then we can completely boot from the system without any operation.

```
setenv bootargs console=ttyS0,115200 ubi.mtd=1 ubi.block=0,4
root=/dev/ubiblock0_4 clk_ignore_unused
setenv bootcmd 'setenv loadaddr 0x22000000; ubi readvol ${loadaddr} kernel
0x8000000; bootm ${loadaddr}#ssd202d-dongshanpione'
```